

Miguel Salvador Francisco Fernandes N° 103024
Pedro Alexandre Ferreira Dias Lopes N° 103194

1 Introduction

For this project we were tasked with creating a simple universe simulator that would allow clients to connect to a server as trash-ships, allowing them to pick up trash in space while everything is being affected by gravity generated from planets.

2 Part A

2.1 Implemented functionalities

Table 1: Implemented functionalities (PART A)

	Not implemented	With faults	Totally Correct
General			
Planets storage			X
Trash storage			X
Trash ship storage			X
Update of screen: planets, trash, ships			X
Universe configurations file			X
Universe-simulator-main			
Trash movement (physics rules)			X
New trash when trash hits planet			X
End of universe			X
Trash ship simulator universe-client			
Connect			X
reads cursor keys			X
sends movement to the server			X
Trash ship simulator universe-server			
Socket creation			X
receives multiple client connections			X
Receives trash ships movement			X
Move the trash ships			X
Trash collected by trash ship			X
Trash ship spill			X

Deposit trash in a recycling planet			X
Misc			
Messages validation			X
Optimization of communication			X
Code organization			X
Code reuse between components			X
Functions return values validation			X

2.2 Description of faulty functionalities

As far as we could tell, nothing has been implemented incorrectly. We are however unsure if rendering the planet names was a requirement, but in any case, they are stored internally within their structure. When we tried to print their names on screen, the application performance downgraded significantly.

2.3 Data types

The universe-simulator contains 3 structs, all within the universe-data.h file:

- The “vector “ struct is used for the calculations of velocity, acceleration and resulting position in the functions provided by the professor. It contains an amplitude and an angle float.
- The planets use the “planet_stucture” struct (we left the typo in the name so that it would match the function provided by the professor). It contains a “char” for it’s name, two floats for the x and y coordinates, an int for mass and an int called “isrecycle” which denotes whether or not it is the recycle planet (0 for regular planets, 1 for the recycle planet)
- The trash uses the “trash_stucture” struct (again, we left the typo for the same reason). It contains two floats for the x and y coordinates, an int for mass, two vectors for it’s velocity and acceleration and an int that denotes its status (status = 0 means the trash is disabled, status = 1 means the trash is enabled)

Additionally, the trashship-simulator contains two more structs:

- The “CommHandle” struct which is used within various communication functions
- The ships use the “trash_ship” struct. It contains two floats for the x and y coordinates, an int called “capacity” that denotes how much trash it’s carrying, and an int for it’s “ID”, which will be unique for each client

2.4 Code organization and Functions List

Many functions are reutilized among the universe and trashship simulators. Most functions contain explanations for their functionality as comments within the code.

For the universe-simulator-main:

In universe-data.c

Programação de Sistemas – 25/26
Project report

- Void universe_data_init - Initializes planets and trash with random positions; randomly picks a recycle planet; "enables" the initial amount of trash.
- Int update_trash_count - Simply counts how many trash are "enabled"

In physics-rules.c

- Make_vector, add_vectors and scale_vector exist to be used in operations within the functions provided by the teacher
- Void correct_position - Function that allows objects to "wrap around" the universe borders
- Void new_trash_acceleration/velocity/position have all been provided by the teacher and remain mostly unchanged
- Void check_collisions - Checks all collisions within the universe: It goes through each trash; if a trash is enabled, it checks it's distance to all planets; if it's less than 1, it "enables" a new trash
- Void physics_update - All encompassing function that updates the physics of the universe

In display.c

- Void universe_display_init - SDL window and renderer initialization
- Void draw_universe - The function that draws the universe renders the planets first, then the trash. This way, the trash will appear "on top" of the trash
- Void destroy_universe - Once the simulation ends, close the SDL window and renderer

Next, for functions unique to the universe-client and universe-server:

In communication.c

- CommHandle *comm_client_init - Create context + REQ socket and connect to server
- int comm_client_send and int comm_client_recv are the corresponding send and recv functions
- CommHandle *comm_server_init and Create context + REP socket and bind on ENDPOINT
- int comm_server_recv and int comm_server_send are the corresponding recv and send functions
- void comm_close - Close socket + context and free handle

In universe-client.c and universe-server.c

- Int read_keys - Read keyboard input and detect when the close button on the window is pressed

In universe-server.c

- Int find_ship_index_by_id - Goes through each position of the ship array to find the index of the ship with the given ID

- `Int allocate_ship_slot` - Finds the first available slot in the ship array, assigns it a random unique ID and gives the ship a random position

2.5 Communication

2.5.1 Sockets

For this part of the project, we only utilized a single REQ/REP socket.

2.5.2 Transferred data

There are two types of messages exchanged:

- The server sends a `ServerMessage`, containing a `ship_id` and a status number. The `ship_id` is used to both differentiate between different clients and randomized to make sure no client can reasonably guess the other client's ID and cheat the system. The status number is used to notify the client of any eventual error
- The client sends a `ClientMessage`, containing a `ship_id` and command. Each number corresponds to a different command. Each arrow key corresponds to a different number being sent.

2.5.3 Error treatment

As far as we are aware, any message sent outside the standard format will be caught and result in an error from either the server or the client.

3 Part B

3.1 Implemented functionalities

Table 2: Implemented functionalities (PART B)

	Not implemented	With faults	Totally Correct
General			
Planets storage			X
Trash storage			X
Trash ship storage			X
Update of screen: planets, trash, ships			X
Universe configurations file			X
new-trash-ship-client			
Connect			X
Reads cursor keys			X
Sends movement to the server			X
Update of screen: planets, trash, ships			X
Detection of server failure			X
new-universe-server			
Initialization of planets and trash			X
Trash movement (physics rules)			X
New trash when trash hits planet			X
Periodic trash generation			X
Interrupts trash generation with no ships			X
End of universe			X
Socket creation			X
Multiple client connections			X
Receives trash ships movement			X
Move the trash ships			X
Gravity and inertia of trash ships			X
Trash collected by trash ship			X
Trash ship spill			X
Trash ship capacity			X
Change of recycling planet			X
Deposit trash in a recycling planet			X
Update of screen: planets, trash, ships			X

new-dashboard			
Planet statistics update			X
Ships statistics update			X
Universe statistics			X
Different programming language			X
Misc			
Messages validation			X
Optimization of communication			X
Code organization			X
Code reuse between client and server			X
Functions return values validation			X

3.2 Description of faulty functionalities

As far as we could tell, nothing has been implemented incorrectly. We are however unsure if rendering the planet names was a requirement, but in any case, they are stored internally within their structure. When we tried to print their names on screen, the application performance downgraded significantly.

3.3 Data types

Only two more structs were added

- The Universe/ClientContext structs are used to pass a multitude of values into new threads.
- An additional “input” parameter was added to the ship struct. It takes note of the current input being done by the client
- An additional “score” parameter was added to the planet struct. It’s sent to the dashboard so it can display the current amount of recycled trash in each planet.

3.4 Code organization and Functions List

Many functions were reutilized/adapted for this part of the project, although some are now in different files. We will only mention completely new ones. Changes for the old ones are typically mentioned within their accompanying comments in the code.

In universe-data.c

- int are_all_ships_empty - Returns 0 if all ship IDs are 0 (there are no connected clients), returns 1 otherwise
- void new_ship_position/velocity/acceleration - Almost identical to the provided functions by the teacher, but used for ships. Only new_ship_acceleration has changed a bit to take into account input from the clients.

In display.c

- void client_display_init – equivalent to void universe_display_init but for clients

In communication.c

- CommHandle *comm_publisher_init - Create context and a PUB socket that will broadcast messages to subscribers
- CommHandle *comm_subscriber_init - Create context and a SUB socket, connect and subscribe to all messages
- Void *input_communication - Function ran by the universe-server thread responsible for handling client inputs and sending the initial universe setup message with a REP socket
- Void send_update - Function used by the server's "broadcast_universe" function; Sends the usual universe-state updates to trashships
- Void send_scoreupdate - Function used by the server's "broadcast_universe" function; Sends the necessary universe info to the dashboard
- Void send_shutdown_message - Function used by the server's "broadcast_universe" function; Alerts the clients of a shutdown, be it by quitting or implosion from too much trash
- Void broadcast_universe - Function ran by the universe-server thread responsible for broadcasting universe state updates to all connected trashship clients, broadcasting universe info to the dashboard and warning clients about server shutdowns, all via a PUB socket
- Void* receive_universe_updates - Function ran by the trashship-client thread responsible for receiving universe state updates from the server via a SUB socket

3.5 Concurrency

3.5.1 Threads

In new-universe-server there are three threads:

- The main thread handles all SDL_events such as the timers responsible for rendering the universe, updating the physics, adding more trash or changing the recycle planets. Also handles the SDL_QUIT event.
- Another thread runs the input_communication function, which will handle communication on a REQ/REP socket and will be responsible for handling inputs from clients
- The other thread runs the broadcast_universe function which will handle communication on a PUB/SUB socket, and will broadcast the universe state to all clients, along with the info for the dashboard

In new-trashship-client there are two threads:

- The main thread handles all SDL_events such as the timer responsible for rendering the universe and any keyboard input events. It also handles communication in the REQ/REP socket, by sending a message containing any new input done by the user

- The other thread runs the `receive_universe_updates` function which will handle the PUB/SUB socket communication, update the universe state and receive any incoming shutdown messages.

3.5.2 Shared variables

Any variable within the `UniverseContext` and `ClientContext` structs is shared and used in someway (although many are only for reading). The more important ones are the `trash`, `ship` and `planet` structures which are frequently accessed by all threads and the `stop_flag` variables, which are used so that the threads are capable of sending a signal to the main thread telling it to shutdown.

3.5.3 Synchronization

Given that the `planet`, `trash` and `ship` structures are frequently accessed by all threads, most operations that read or write these structures are protected by a mutex. There are too many cases to take note of here, but most of them happen within the `new-universe-server.c` and `new-trashship-client.c` files.

3.6 Communication

3.6.1 Sockets

In addition to the original REQ/REP socket that handles input communication and sends the initial universe state to the clients, there is now also a PUB/SUB socket, which is used to broadcast the universe-state to all the clients, along with broadcasting the necessary info for the dashboard, or any incoming shutdown messages.

3.6.2 Transferred data

In addition to the original message types, newer message types are used mostly by the new PUB/SUB socket.

- `InitialSetupMessage` is sent by the REQ/REP socket and contains information on the coordinates of all planets, trash and ships, as well as information on which planet is the recycle one.
- After that first message, every other universe state is sent in the `UpdateMessage` type, which avoids sending the coordinates for the planets as those never change. It does however also include a status parameter, warning the client of any shutdown.
- Finally, the `ScoreUpdate` type is used to send the information required by the dashboard.

3.6.3 Error treatment

As far as we are aware, any message sent outside the standard format will be caught and result in an error from either the server or the client. Proper timeouts were implemented to take into account unordered exits from the server or clients.

4 Conclusion

This project allowed us to better understand client-server architectures, socket-based communication, and concurrent programming with shared data. We successfully implemented a complete distributed simulation with physics, real-time updates, and multiple interacting components. The main challenges were handling synchronization between threads and optimizing communication without impacting performance. Overall, the project was a valuable learning experience that reinforced both system design and low-level programming concepts.